

Link to code: <https://github.com/luca-miniati/ds4400/blob/main/hw2/hw2.ipynb>

Solution 1

1. $\theta_1 = \rho_{\frac{\sigma_Y}{\sigma_X}} = 1.6$
 $\theta_0 = \bar{Y} - \theta_1 \bar{X} = 20$
 $h_\theta(x) = 20 + 1.6x$
2. $\theta_1 = \rho_{\frac{\sigma_Y}{\sigma_X}} = -1.6$
 $\theta_0 = \bar{Y} - \theta_1 \bar{X} = 180$
 $h_\theta(x) = 180 - 1.6x$

Solution 2

1. Coefficients:

feature	coefficient
sqft_basement	0.07997505949865752
sqft_living	0.16724347457887279
yr_renovated	0.05090017146159664
sqft_lot	0.03206975845539397
grade	0.27181371785441855
yr_built	-0.199349812705525
sqft_above	0.14231485091519286
long	-0.0030508283583896368
lat	0.23097972233559538
sqft_lot15	-0.038106038178795994
view	0.14204967148330078
sqft_living15	0.13432109423600497
floors	0.02370550476678393
bedrooms	-0.03690324823375112
bathrooms	0.05460245199491633
condition	0.03820676468517668
waterfront	0.1878555509336244

Training MSE: 0.27319310156126875

Training R^2 : 0.7265334318706018

2. Testing MSE: 0.47014226635183515
Testing R^2 : 0.6750073253711697
3. The most significant features are **grade**, **lat**, **yr_built**, **waterfront**, **sqft_living**, **sqft_above**, and **view**. Since the R^2 is pretty high, the features are correlated with the price. The on the test set, MSE increases slightly and the R^2 decreases slightly. It doesn't look like the model is overfitting.

Solution 3

1. Linear Regression MSE: 0.30846451541843545
 Linear Regression R^2 : 0.6912267112928574
 The results look pretty similar between the `sklearn` implementation and my implementation.

Solution 4

1. Polynomial Regression Train MSE ($p = 1$): 0.5027879071385489
 Polynomial Regression Train R^2 ($p = 1$): 0.49670880166311404
 Polynomial Regression Test MSE ($p = 1$): 0.7685389493673271
 Polynomial Regression Test R^2 ($p = 1$): 0.46873628136126444

Polynomial Regression Train MSE ($p = 2$): 0.47567471613627627
 Polynomial Regression Train R^2 ($p = 2$): 0.5238491329967203
 Polynomial Regression Test MSE ($p = 2$): 0.622908183771123
 Polynomial Regression Test R^2 ($p = 2$): 0.5694056646664778

Polynomial Regression Train MSE ($p = 3$): 0.46667299327457745
 Polynomial Regression Train R^2 ($p = 3$): 0.5328598665920146
 Polynomial Regression Test MSE ($p = 3$): 0.866215897185062
 Polynomial Regression Test R^2 ($p = 3$): 0.401215671552679

Polynomial Regression Train MSE ($p = 4$): 0.45808818520169264
 Polynomial Regression Train R^2 ($p = 4$): 0.5414532680663737
 Polynomial Regression Test MSE ($p = 4$): 2.1776485108552395
 Polynomial Regression Test R^2 ($p = 4$): -0.5053311829119937

It looks like the model for $p = 1$ was the best. As p increases, the Test MSE gets huge and the Test R^2 gets tiny, indicating overfitting.

Solution 5

1. Gradient Descent (lr = 0.01, iterations = 10) Train MSE: 0.47671717428523064
 Gradient Descent (lr = 0.01, iterations = 10) Train R^2 : 0.5228056313461155
 Gradient Descent (lr = 0.01, iterations = 10) Test MSE: 0.7894754680802667
 Gradient Descent (lr = 0.01, iterations = 10) Test R^2 : 0.4542636085111217

Gradient Descent (lr = 0.01, iterations = 50) Train MSE: 0.293333974988953
 Gradient Descent (lr = 0.01, iterations = 50) Train R^2 : 0.7063723974084555
 Gradient Descent (lr = 0.01, iterations = 50) Test MSE: 0.5338535693382458
 Gradient Descent (lr = 0.01, iterations = 50) Test R^2 : 0.6309659612064211

Gradient Descent (lr = 0.01, iterations = 100) Train MSE: 0.27815604191853505
 Gradient Descent (lr = 0.01, iterations = 100) Train R^2 : 0.72156552360507
 Gradient Descent (lr = 0.01, iterations = 100) Test MSE: 0.5119104782807328

Gradient Descent ($lr = 0.01$, iterations = 100) Test R^2 : 0.6461344418191988

Gradient Descent ($lr = 0.1$, iterations = 10) Train MSE: 0.27750955904693375

Gradient Descent ($lr = 0.1$, iterations = 10) Train R^2 : 0.7222126536066729

Gradient Descent ($lr = 0.1$, iterations = 10) Test MSE: 0.5108641310763316

Gradient Descent ($lr = 0.1$, iterations = 10) Test R^2 : 0.6468577445317745

Gradient Descent ($lr = 0.1$, iterations = 50) Train MSE: 0.27328973417390096

Gradient Descent ($lr = 0.1$, iterations = 50) Train R^2 : 0.7264367025286276

Gradient Descent ($lr = 0.1$, iterations = 50) Test MSE: 0.500837856220127

Gradient Descent ($lr = 0.1$, iterations = 50) Test R^2 : 0.6537885527472672

Gradient Descent ($lr = 0.1$, iterations = 100) Train MSE: 0.27319540933538733

Gradient Descent ($lr = 0.1$, iterations = 100) Train R^2 : 0.726531121786399

Gradient Descent ($lr = 0.1$, iterations = 100) Test MSE: 0.5001109337166414

Gradient Descent ($lr = 0.1$, iterations = 100) Test R^2 : 0.6542910484928399

Gradient Descent ($lr = 0.5$, iterations = 10) Train MSE: 1232464713720.624

Gradient Descent ($lr = 0.5$, iterations = 10) Train R^2 : -1233698412131.7568

Gradient Descent ($lr = 0.5$, iterations = 10) Test MSE: 1376361686007.4016

Gradient Descent ($lr = 0.5$, iterations = 10) Test R^2 : -951430019391.141

Gradient Descent ($lr = 0.5$, iterations = 50) Train MSE: 9.655673783656819e+61

Gradient Descent ($lr = 0.5$, iterations = 50) Train R^2 : -9.665339122779599e+61

Gradient Descent ($lr = 0.5$, iterations = 50) Test MSE: 1.07830247626251e+62

Gradient Descent ($lr = 0.5$, iterations = 50) Test R^2 : -7.453922586853501e+61

Gradient Descent ($lr = 0.5$, iterations = 100) Train MSE: 2.250570894948878e+124

Gradient Descent ($lr = 0.5$, iterations = 100) Train R^2 : -2.252823718667546e+124

Gradient Descent ($lr = 0.5$, iterations = 100) Test MSE: 2.513336949240457e+124

Gradient Descent ($lr = 0.5$, iterations = 100) Test R^2 : -1.7373806948168516e+124

At $lr = 0.01$, the model converged steadily. At $lr = 0.1$, the model converged faster than $lr = 0.01$, and seems to be the best choice (since the training wasn't unstable). At $lr = 0.5$, the training was completely unstable and didn't converge at all.

Solution 6

1.

$$\begin{aligned}J(\theta) &= (X\theta - y)^T(X\theta - y) + \lambda\theta^T\theta \\&= (\theta^T X^T - y^T)(X\theta - y) + \lambda\theta^T\theta \\&= \theta^T X^T X\theta - y^T X\theta - y\theta^T X^T + y^T y + \lambda\theta^T\theta \\&= \theta^T X^T X\theta - 2X^T y\theta + y^T y + \lambda\theta^T\theta \\\nabla J &= 2X^T X\theta - 2X^T y + 2\lambda\theta \\0 &= 2X^T X\theta - 2X^T y + 2\lambda\theta \\2X^T y &= 2(X^T X + I\lambda)\theta \\\theta &= (X^T X + I\lambda)^{-1}X^T y\end{aligned}$$

2. Ridge GD (lr = 0.001, iterations = 100, lambda = 1) Slope: 0.4133560043771
Ridge GD (lr = 0.001, iterations = 100, lambda = 1) Train MSE: 8.028257679187
Ridge GD (lr = 0.001, iterations = 100, lambda = 1) Train R2: 0.12237712347637

Ridge GD (lr = 0.001, iterations = 100, lambda = 10) Slope: 0.20473805481084
Ridge GD (lr = 0.001, iterations = 100, lambda = 10) Train MSE: 9.102396404485
Ridge GD (lr = 0.001, iterations = 100, lambda = 10) Train R2: 0.004955790535624

Ridge GD (lr = 0.001, iterations = 100, lambda = 100) Slope: 0.025457336370211
Ridge GD (lr = 0.001, iterations = 100, lambda = 100) Train MSE: 10.127288241922
Ridge GD (lr = 0.001, iterations = 100, lambda = 100) Train R2: -0.10708203366488

Ridge GD (lr = 0.001, iterations = 100, lambda = 1000) Slope: -0.000780054016390
Ridge GD (lr = 0.001, iterations = 100, lambda = 1000) Train MSE: 10.2845885701
Ridge GD (lr = 0.001, iterations = 100, lambda = 1000) Train R2: -0.12427759115880

Ridge GD (lr = 0.001, iterations = 100, lambda = 10000) Slope: -1.9628230886330e+124
Ridge GD (lr = 0.001, iterations = 100, lambda = 10000) Train MSE: 5.724017490540e+248
Ridge GD (lr = 0.001, iterations = 100, lambda = 10000) Train R2: -6.257308741276e+247

It looks like the lambda parameter is not helping. As it gets bigger, training becomes more unstable. At lambda = 10000, the algorithm diverges.